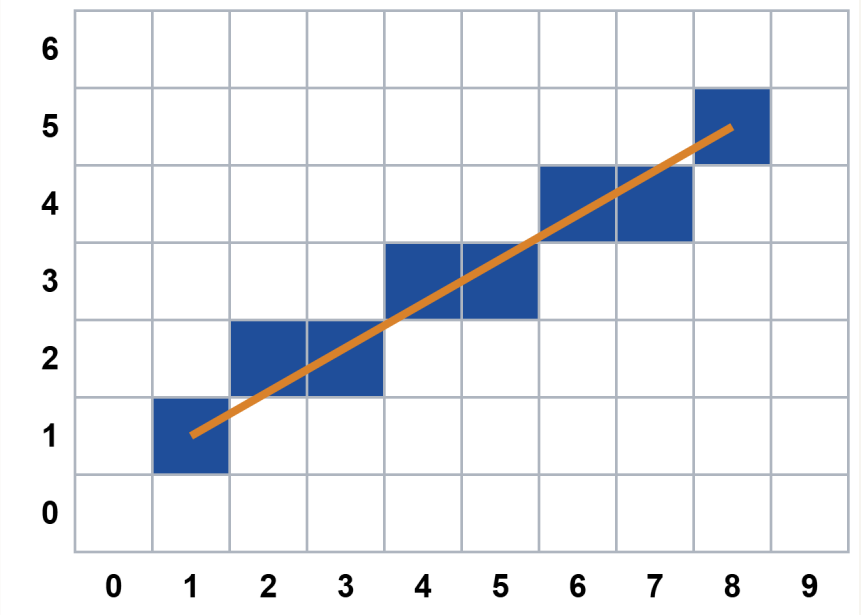
0613-403 · Week 2 · Lecture 4

Bresenham's Line Algorithm

Ekramul Haque Amin

Lecturer, AI and Data Science

Dhaka International University



Warm-up: DDA in one minute

Q1

For (2, 2) to (8, 5), what are steps, xInc and yInc?

Q2

Which pixel does DDA plot when y = 3.5?

Q3

Name two weaknesses of DDA.

Today we remove every one of those weaknesses.

Today's plan

Part 1

Bresenham

- The idea
- Every variable
- Full proof of each formula
- Algorithm and examples
- Steep lines and code

Part 2

Worked examples

- Two examples step by step
- Class exercise
- A steep line (m > 1)
- Practice questions

Part 1

Bresenham's line algorithm

Choosing the next pixel with whole numbers only

Why Bresenham?

DDA

y = 9 → 9.6 → 10.2 → 10.8 ...

- **Floating-point additions**
- **A rounding at every step**
- **Errors can build up on long lines**

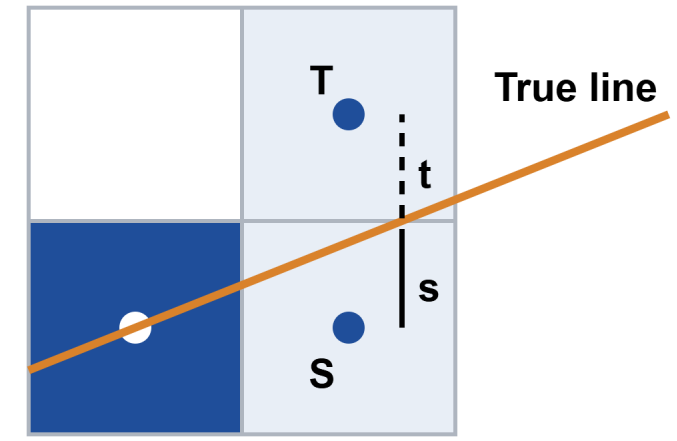
Bresenham (1965)

Asks a yes/no question at every step.

- **Only integer addition and subtraction**
- **No rounding at all**
- **Built into graphics hardware**

Key idea: at each step only two pixels are possible. Choose the one nearer the true line, using a whole-number test.

Only two choices at each step



Assume 0 ≤ m ≤ 1 and x1 < x2, so x always goes up by exactly 1.

$S = (x_i + 1, y_i)$

Right: y stays the same

$T = (x_i + 1, y_i + 1)$

Right and one up

Pick the one closer to the orange line.

Variables and their meaning (1): the line

Symbol	Meaning
(x1, y1)	Start point of the line (whole numbers)
(x2, y2)	End point of the line
dx = x2 − x1	Horizontal distance (always > 0 here)
dy = y2 − y1	Vertical distance
m = dy ÷ dx	Slope of the line, 0 ≤ m ≤ 1 for this case
b	y-intercept: where the line meets the y-axis
y = mx + b	Equation of the true (mathematical) line

Variables and their meaning (2): the decision

Symbol	Meaning
(x_i, y_i)	The pixel just plotted at step i
S, T	The two candidate pixels: right, or right-and-up
y	True line height at $x = x_i + 1$ (not a whole number)
s, t	Distance from S up to the line, and from the line up to T
d_i	Decision parameter at step i: its sign tells S or T
c	A constant that does not change from step to step
inc1 = 2dy, inc2 = 2(dy – dx)	Amounts added to d after choosing S or T

Proof step 1: the two distances

The true line at $x = x_i + 1$:

$y = m(x_i + 1) + b$ ... (1)

Distance from S up to the line:

$s = y - y_i$ ... (2)

Distance from the line up to T:

$t = (y_i + 1) - y$ ... (3)

Meaning

If s is smaller, the line is closer to S.

If t is smaller, the line is closer to T.

So we only need the sign of s – t.

Proof step 2: the difference s − t

$$s - t = (y - y_i) - ((y_i + 1) - y)$$

$$= y - y_i - y_i - 1 + y$$

$$= 2y - 2y_i - 1$$

Put in y from (1):

$$s - t = 2m(x_i + 1) + 2b - 2y_i - 1 \ldots (4)$$

Rule

$s - t < 0 \rightarrow s < t \rightarrow$ choose S

$s - t \geq 0 \rightarrow s \geq t \rightarrow$ choose T

(The old slide wrote “s < t” here by mistake.)

Proof step 3: remove the fraction

$m = dy \div dx$ is a fraction. Multiply (4) by dx:

$d_i = dx(s - t)$

$= dx[2(dy/dx)(x_i + 1) + 2b - 2y_i - 1]$

$= 2dy \cdot x_i + 2dy + 2dx \cdot b - 2dx \cdot y_i - dx$

$d_i = 2dy \cdot x_i - 2dx \cdot y_i + c,\ c = 2dy + dx(2b - 1)$

Why this is allowed

$dx > 0$, so multiplying by dx never changes the sign.

d_i has the same sign as $s - t$, so the rule stays: $d_i < 0 \rightarrow S$, $d_i \geq 0 \rightarrow T$.

c is the same at every step.

Proof step 4: the update rule

Next step: $d_{i+1} = 2dy \cdot x_{i+1} - 2dx \cdot y_{i+1} + c$

Subtract d_i:

$d_{i+1} - d_i = 2dy(x_{i+1} - x_i) - 2dx(y_{i+1} - y_i)$

x always goes up by 1, so $x_{i+1} - x_i = 1$:

$$d_{i+1} = d_i + 2dy - 2dx(y_{i+1} - y_i) \quad \ldots (5)$$

Notice

c has cancelled out.

We never need b or m again.

Only $y_{i+1} - y_i$ is left to decide: it is 0 or 1.

Proof step 5: the two cases

Chose S ($d_i < 0$)

$y_{i+1} = y_i$, so $y_{i+1} - y_i = 0$

$$d_{i+1} = d_i + 2dy$$

Chose T ($d_i \geq 0$)

$y_{i+1} = y_i + 1$, so $y_{i+1} - y_i = 1$

$$d_{i+1} = d_i + 2dy - 2dx$$

Names used in code

inc1 = 2dy

inc2 = 2(dy – dx)

Both are worked out once, before the loop.

Proof step 6: the starting value d_1

At the first pixel (x1, y1), use (4):

$s - t = 2m(x1 + 1) + 2b - 2y1 - 1$

$= 2(m \cdot x1 + b - y1) + 2m - 1$

(x1, y1) is on the line, so $m \cdot x1 + b - y1 = 0$:

$s - t = 2m - 1$

$d_1 = dx(2m - 1) = dx(2 \cdot dy/dx - 1)$

$$d_1 = 2dy - dx$$

Result

The start value uses only dx and dy.

No fractions, no b.

Bresenham formula card

Start

$$d_1 = 2dy - dx$$

Proof step 6

If $d_i < 0 \rightarrow$ plot S

$$d_{i+1} = d_i + 2dy$$

y stays · Proof step 5

If $d_i \geq 0 \rightarrow$ plot T

$$d_{i+1} = d_i + 2(dy - dx)$$

y + 1 · Proof step 5

d_i is the current decision value and d_{i+1} is the next one. Every step: x + 1. Only integers, one comparison and one addition per pixel.

Bresenham's algorithm ($0 \le m \le 1$)

1. Input (x1, y1) and (x2, y2) with x1 < x2.
2. dx = x2 − x1, dy = y2 − y1
3. $d_1 = 2dy - dx$, inc1 = 2dy, inc2 = 2(dy − dx)
4. x = x1, y = y1. Plot (x, y).
5. While x < x2: if $d_i < 0$ then $d_{i+1} = d_i + inc1$; else $d_{i+1} = d_i + inc2$ and y = y + 1.
6. x = x + 1. Plot (x, y). Repeat step 5.

Exam table

Columns: x, y, d, decision, new d.

Check: dx + 1 pixels; last pixel = (x2, y2).

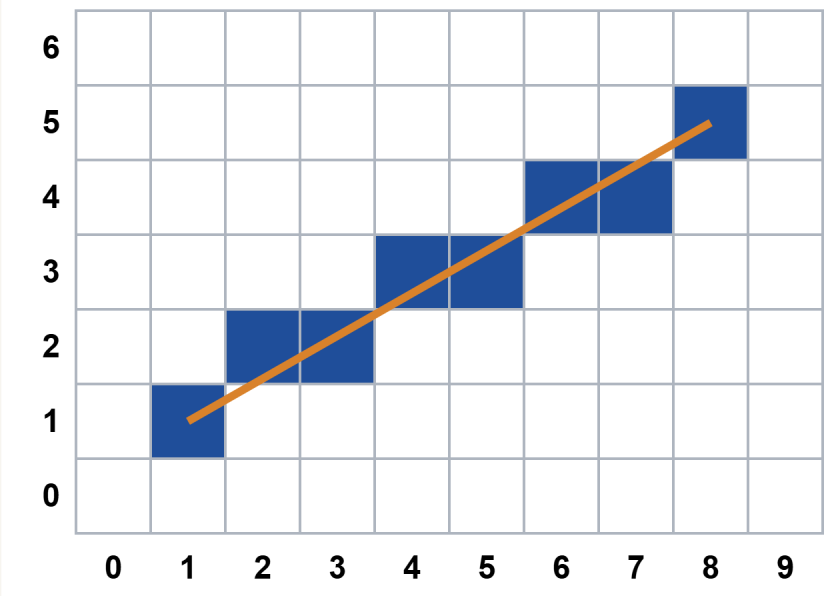
Bresenham in C++ with OpenGL

```
void bresenhamLine(int x1, int y1, int x2, int y2) {
    int dx = x2 - x1, dy = y2 - y1;          // 0 ≤ m ≤ 1
    int d = 2 * dy - dx;                      // d1
    int inc1 = 2 * dy, inc2 = 2 * (dy - dx);
    int x = x1, y = y1;
    glBegin(GL_POINTS);
    glVertex2i(x, y);
    while (x < x2) {
        if (d < 0) d += inc1;                 // choose S
        else { d += inc2; y++; }              // choose T
        x++;
        glVertex2i(x, y);
    }
    glEnd();
}
```

Example 1: line from (1, 1) to (8, 5)

dx = 7, dy = 4, $d_1 = 2(4) - 7 = 1$, inc1 = 8, inc2 = −6

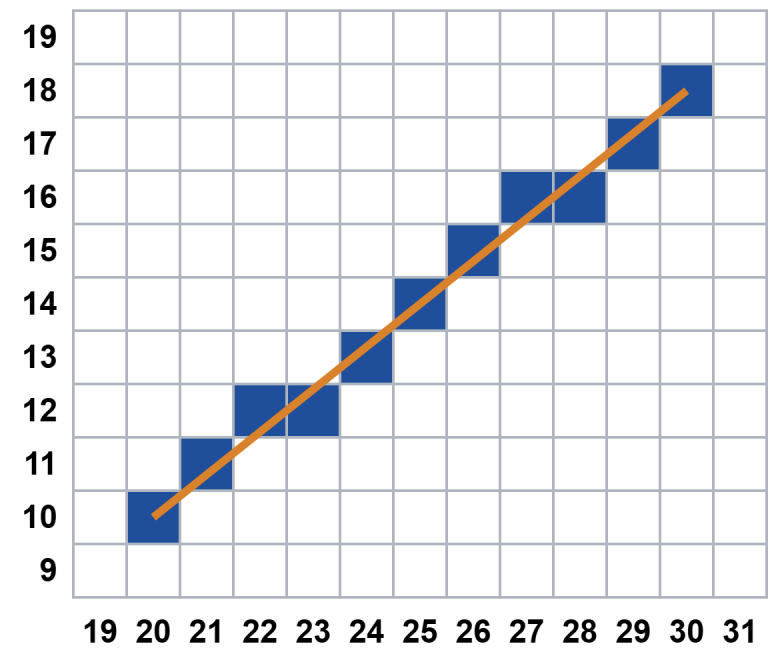
x	y	d	Decision	New d
1	1	1	d ≥ 0 → T, y + 1	-5
2	2	-5	d < 0 → S, y stays	3
3	2	3	d ≥ 0 → T, y + 1	-3
4	3	-3	d < 0 → S, y stays	5
5	3	5	d ≥ 0 → T, y + 1	-1
6	4	-1	d < 0 → S, y stays	7
7	4	7	d ≥ 0 → T, y + 1	1
8	5	—	Stop: reached (x2, y2)	—



Example 2: line from (20, 10) to (30, 18)

dx = 10, dy = 8, d_1 = 6, inc1 = 16, inc2 = −4

x	y	d	Decision	New d
20	10	6	d ≥ 0 → T, y + 1	2
21	11	2	d ≥ 0 → T, y + 1	-2
22	12	-2	d < 0 → S, y stays	14
23	12	14	d ≥ 0 → T, y + 1	10
24	13	10	d ≥ 0 → T, y + 1	6
25	14	6	d ≥ 0 → T, y + 1	2
26	15	2	d ≥ 0 → T, y + 1	-2
27	16	-2	d < 0 → S, y stays	14
28	16	14	d ≥ 0 → T, y + 1	10
29	17	10	d ≥ 0 → T, y + 1	6
30	18	—	Stop: reached (x2, y2)	—

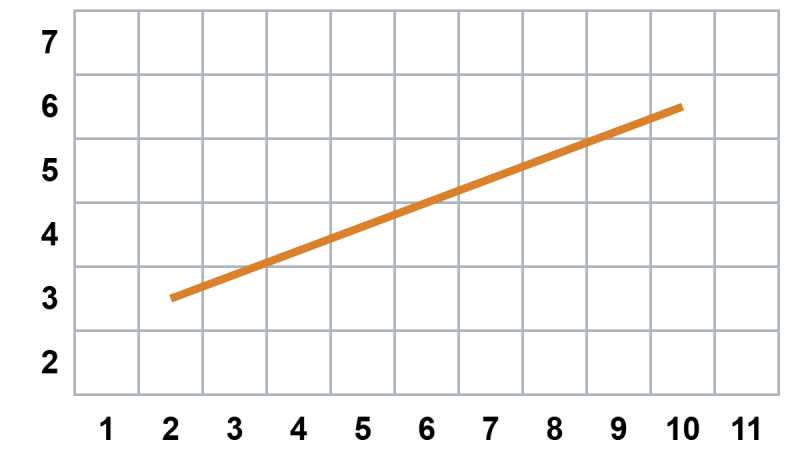


Class exercise: your turn

Use Bresenham to scan convert the line from (2, 3) to (10, 6).

1. Find dx, dy, d_1, inc1, inc2
2. Fill in the table
3. Shade the pixels

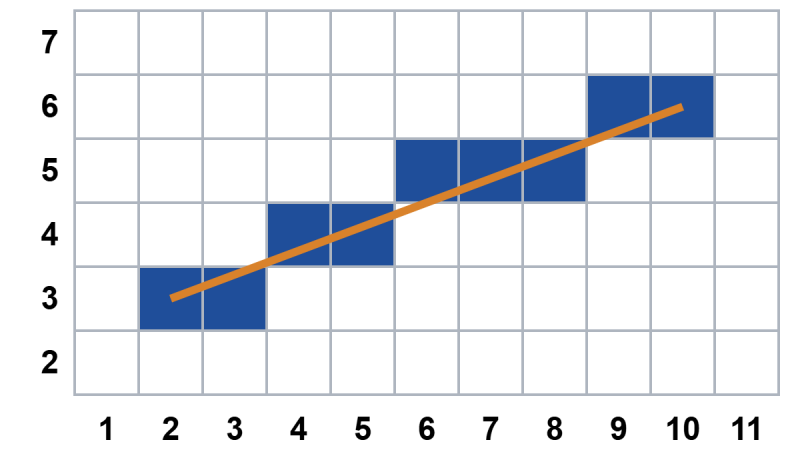
Time: 5 minutes.



Exercise answer: (2, 3) to (10, 6)

dx = 8, dy = 3, $d_1 = -2$, inc1 = 6, inc2 = −10

x	y	d	Decision	New d
2	3	-2	d < 0 → S, y stays	4
3	3	4	d ≥ 0 → T, y + 1	-6
4	4	-6	d < 0 → S, y stays	0
5	4	0	d ≥ 0 → T, y + 1	-10
6	5	-10	d < 0 → S, y stays	-4
7	5	-4	d < 0 → S, y stays	2
8	5	2	d ≥ 0 → T, y + 1	-8
9	6	-8	d < 0 → S, y stays	-2
10	6	—	Stop: reached (x2, y2)	—



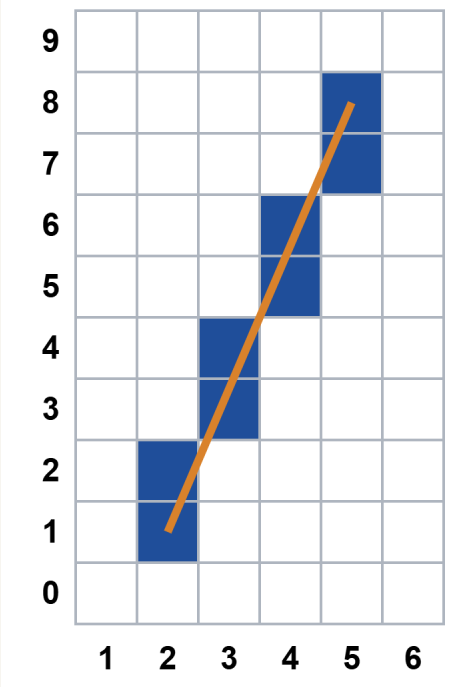
Steep lines (m > 1): swap the roles of x and y

y goes up by 1 every step; d decides whether x goes up.

$d_1 = 2dx - dy$, inc1 = 2dx, inc2 = 2(dx – dy)

Example (2, 1) to (5, 8): dx = 3, dy = 7, $d_1 = -1$

x	y	d	Decision	New d
2	1	-1	d < 0 → x stays	5
2	2	5	d ≥ 0 → x + 1	-3
3	3	-3	d < 0 → x stays	3
3	4	3	d ≥ 0 → x + 1	-5
4	5	-5	d < 0 → x stays	1
4	6	1	d ≥ 0 → x + 1	-7
5	7	-7	d < 0 → x stays	-1
5	8	—	Stop: reached (x2, y2)	—



Key points from today

1. Bresenham chooses between S (right) and T (right and up) at each step.
2. $d_i = dx(s - t)$ has the same sign as $s - t$, because $dx > 0$.
3. $d_1 = 2dy - dx$; if $d_i < 0$, $d_{i+1} = d_i + 2dy$; if $d_i \geq 0$, $d_{i+1} = d_i + 2(dy - dx)$ and y goes up by 1.
4. Only integers: no rounding, no build-up of error.
5. For $m > 1$, swap the roles of x and y.
6. It needs only one comparison and one addition per pixel.

Practice questions

1. Use Bresenham to find the pixels of the line from (0, 0) to (8, 6).
2. Use Bresenham to find the pixels of the line from (5, 5) to (13, 9).
3. Use Bresenham to find the pixels of the steep line from (1, 1) to (4, 7).
4. Why do we multiply $s - t$ by dx? Why does this not change the decision?
5. Derive the starting value $d_1 = 2dy - dx$.
6. Use Bresenham to find the pixels of the line from (2, 2) to (10, 7).

Coming up

Next lecture

Midpoint line algorithm and comparison

The midpoint test, why it gives the same formulas as Bresenham, and direct vs DDA vs Bresenham

Lab

Lab 2: DDA and Bresenham

Code both algorithms in Python with PyOpenGL and compare the lines they draw

Questions?

Thank you

Ekramul Haque Amin · Lecturer, AI and Data Science